

Dimensional modelling and ETL data processing

Professor: Anália Lourenço
Students: Azaliia Agisheva,
José Manuel Bueno Tabernero

Data Engineering
Master in Artificial Intelligence

Table of contents

[Project Objective](#)

[Dimensional modelling](#)

[Analytical Goals and Models](#)

[Qualifying model](#)

[Pit stops model](#)

[Race final positions model](#)

[Why these dimensional models](#)

[Data Model Highlights](#)

[ETL data processing](#)

[Pipeline Overview](#)

[Flow Diagram](#)

[Star Schemas \(Grain And Keys\)](#)

[Important Semantics And Sources](#)

[Observed Statistics During Load](#)

[ETL Load Results \(Per Schema\)](#)

[Why some race_result records were skipped](#)

[How To Run](#)

[References](#)

[Data analysis](#)

Project Objective

Model driver performance during qualifying sessions and races (pit stops and final positions) using three star schemas and a unified ETL pipeline that reads CSV data, cleans and transforms it, and loads curated facts and dimensions into MySQL. Design and implement an analytical report in Tableau that describes the main analytical goals of the project.

Dimensional modelling

Analytical Goals and Models

Qualifying model

- Rank drivers by one-lap speed using Q1/Q2/Q3 times and the final qualifying position.
- Check consistency by looking at how much a driver's positions and times vary across a season.
- Measure progress within a session: time changes from Q1 → Q2 → Q3 for each driver.
- Calculate the gap to pole in seconds and as a percentage for each driver and team.
- Track how often drivers and teams reach Q3 and how often poles turn into wins by season.
- Compare teammates on average position, Q3 reach rate, and gap to pole.
- Separate driver skill from team effect and measure spread inside each team.
- See how circuits matter: rank drivers and teams by circuit and country.
- Find "driver-circuit" strengths and weaknesses.
- Monitor reliability: DNQ/DNS/DSQ rates by driver, team, and circuit.

- Study trends over time in pace, gaps, and overall competitiveness.
- Test time-of-day effects using `dim_date.time_bucket` (morning/afternoon/evening).
- Use `raceld` to group sessions and compare multi-driver profiles for the same event.
- Flag outliers such as unusually slow laps or sudden drops vs recent form.

Pit stops model

Star schema adds the race dimension to analyze event-level operations and lap details. The pit stop schema omits the constructor dimension because `pit_stops.csv` does not contain constructor identifiers.

- Rank drivers by average pit-stop time per race and per season.
- Check consistency: variance of pit times by driver, team, and year.
- Measure within-race improvement: time deltas between a driver's stops.
- Compare teammates on pit-stop time and number of stops.
- Compute "clean pit" rate: share of stops under set thresholds (e.g., <2.5s, <3.0s).
- Analyze lap context: distribution of pit times by lap number and stint phase.
- Link strategy load to outcome: stops per driver vs next-lap position (if available later).
- Identify outliers: very slow stops and how often they happen by driver and team.
- Track reliability: consecutive races without slow or failed stops by team.
- Measure circuit effect: pit-lane performance by circuit and country.
- Study trends: year-over-year changes in median pit time by team and driver.
- Examine undercut/overcut windows using `duration_ms` as the loss proxy vs race pace.
- Profile timing around Safety Car/Virtual Safety Car windows (if flags are added later).

- Build a Pit-Stop Efficiency Index combining median time, variability, and clean-pit rate.

Race final positions model

Star schema anchors results on the race dimension with season and round attributes.

- Rank drivers by average finish, win rate, podium rate, and points per start.
- Check consistency: standard deviation of finishing positions by driver and season.
- Compute positions gained or lost vs grid, per race and per season.
- Conversion metrics: pole→win, top-3 grid→podium, top-10 grid→points.
- Recovery index: points finishes from starts outside the top 10.
- Clutch finishes: very small final gaps using `race_time_ms` where available.
- Teammate comparison: average finish, points, and positions gained.
- Reliability: DNF/DNS/DSQ via `status_text` ; classified finish rate.
- Laps durability: percent of laps completed and average laps by driver and team.
- Pace vs result: correlation between fastest-lap rank/speed and final position.
- Start effectiveness: grid deciles vs median finishing position.
- Circuit effect: positions gained and driver rankings by circuit and country.
- Constructor effect: team-adjusted driver performance and intra-team spread.
- Trends over time: year-over-year changes in finishes and points.
- Pressure handling: variance in finishes in late-season rounds vs early rounds.
- Composite "Race Result Index" combining finish, positions gained, points, and reliability.

Why these dimensional models

- Surrogate keys on all dimensions protect analytics from raw CSV IDs, support stable joins, history tracking, and indexing.

- Qualifying analysis needs circuit, constructor, driver, and date. A separate race dimension is not required because qualifying date (usually `race_date - 1`) gives enough event context.
- Pit stops and race results need the race dimension to capture per-race behavior, enforce the race-driver grain, and keep key attributes like year, round, and start time.

Data Model Highlights

- No original CSV IDs are stored in any schema. All dimensions and facts use surrogate keys generated by AUTO_INCREMENT (date uses a smart surrogate `date_key` of form YYYYMMDD). Original IDs are only used transiently during transformation to build lookups.
- Lookup mapping: After dimensions load, CSV IDs are mapped to AUTO_INCREMENT keys via in-memory lookup tables, then applied to facts to populate foreign keys. This avoids carrying CSV IDs into the warehouse.
- Qualifying facts include an additional derived column `status` to classify outcomes:

`/* status — Derived field indicating the driver's result in the qualifying session. It is not present in the original dataset, but calculated from the availability of q1, q2, and q3 times:`

`'OK' — the driver set times in all qualifying rounds (Q1–Q3).`

`'DNQ' — the driver participated in Q1 but did not advance to Q2.`

`'DNS' — the driver did not start the qualifying session (no times recorded).`

`'DSQ' — the driver was disqualified or had no valid times due to penalties.`

`This column helps categorize the performance outcome of each driver during qualification. */`

Status Derivation

- 'OK': Has Q1, Q2, and Q3 times
- 'DNQ': Has Q1, missing Q2/Q3 (Did Not Qualify)
- 'DNS': No times recorded (Did Not Start)
- 'DSQ': Invalid/penalty times (Disqualified)

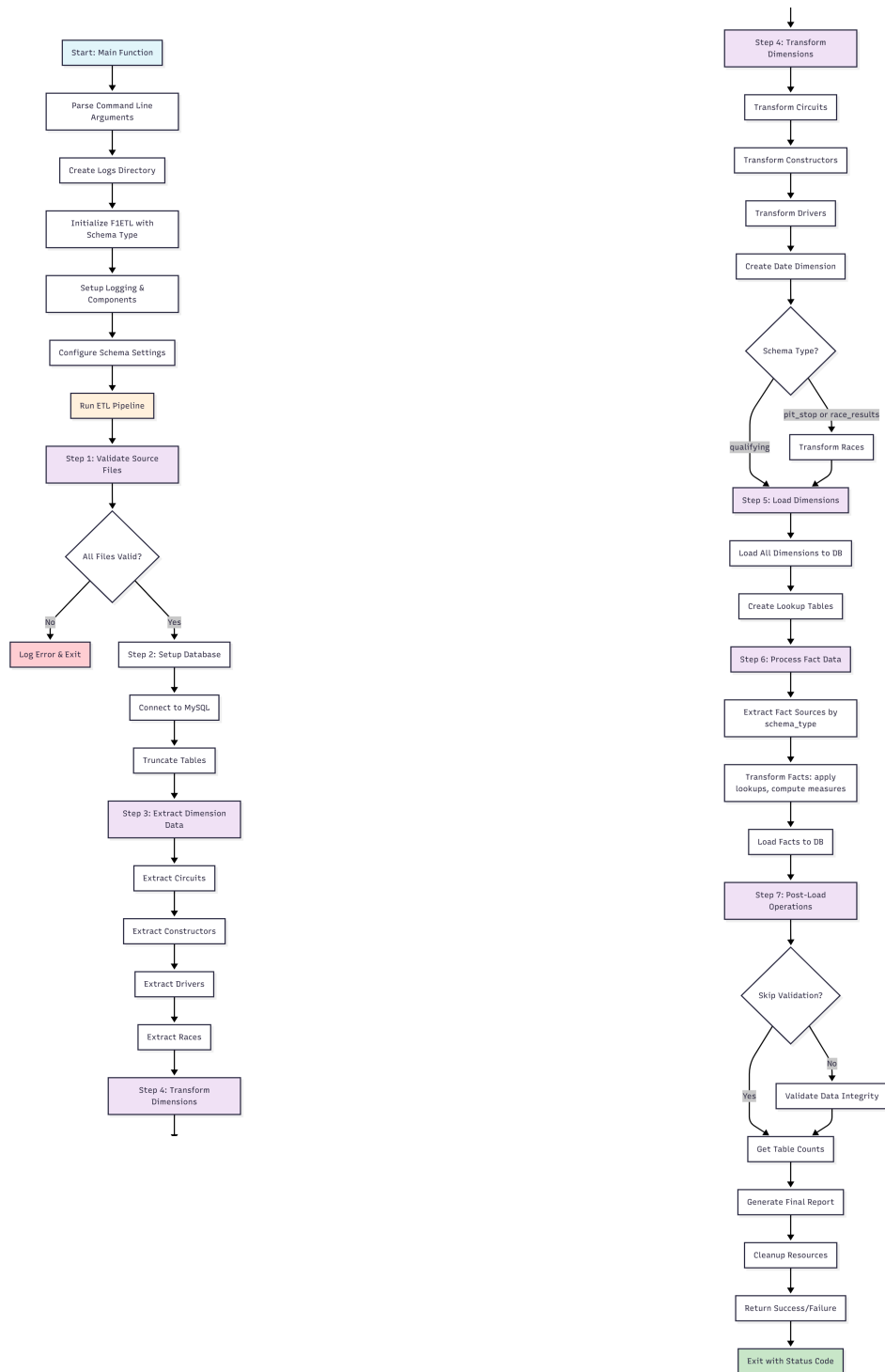
ETL data processing

Pipeline Overview

High-level steps common to all three schemas:

- Source read: Load core CSVs (`circuits.csv` , `constructors.csv` , `drivers.csv` , `racers.csv`) and the schema-specific file(s):
 - Qualifying: `qualifying.csv`
 - Pit stops: `pit_stops.csv`
 - Race results: `results.csv` plus `status.csv` (status text lookup)
- Clean and standardize: Enforce dtypes, strip names, validate required columns, drop rows missing critical references (race/driver/constructor as applicable).
- Transform dimensions: Prepare standardized dimension tables (circuit, constructor, driver, date; and race for pit_stop/race_results). Date logic differs per schema: qualifying uses `race_date - 1`; pit_stop/race_results use `race_date`.
- Load dimensions: Insert to MySQL, create CSV_ID → surrogate key lookups. Surrogate keys are AUTO_INCREMENT (date uses the derived `date_key`).
- Transform facts: Apply lookups to add FKs, compute measures, and select final grain:
 - Qualifying: convert Q1/Q2/Q3 strings to milliseconds; derive `status` ; link to circuit/constructor/driver/date.
 - Pit stops: derive `stop_number` , `lap` , and `duration_ms` ; link to race/circuit/driver/date.
 - Race results: join `results.csv` with `status.csv` to populate `status_text` ; compute grid/finish/laps/fastest-lap measures; link to race/circuit/constructor/driver/date; de-duplicate race–driver pairs.
- Load facts: Append batches to the target fact table for each schema; optional integrity validation runs post-load.

Flow Diagram



Comments on the flow:

- Source validation halts early on missing/invalid files, avoiding partial loads.
- Dimensions load first to establish surrogate keys; lookups are then applied to facts.
- Qualifying reads in batches for scalability; pit_stop and race_results process fully.
- Post-load validation checks FKs, ranges, and business consistency before reporting.

Star Schemas (Grain And Keys)

- Qualifying schema (database: `f1_qlf_db`)
 - Dimensions: `dim_circuit`, `dim_constructor`, `dim_driver`, `dim_date` (surrogate keys; date uses `date_key`)
 - Fact: `facts` at the driver-in-event (qualifying attempt) grain with measures `position`, `q1_ms`, `q2_ms`, `q3_ms`, and derived `status`.
- Pit stop schema (database: `f1_pit_db`)
 - Dimensions: `dim_race`, `dim_circuit`, `dim_driver`, `dim_date` (all surrogate keys)
 - Fact: `fact_pit_stop` at driver-race-stop grain with `stop_number`, `lap`, `duration_ms`.
- Race results schema (database: `f1_res_db`)
 - Dimensions: `dim_race`, `dim_circuit`, `dim_constructor`, `dim_driver`, `dim_date` (all surrogate keys)
 - Fact: `fact_race_result` at driver-race grain with grid/finish/points/laps/fastest-lap metrics and `status_text`.

Why we added a race dimension for pit stops and final positions:

- Pit stops and final positions are inherently anchored to the race event. The `dim_race` preserves year, round, Grand Prix name, and start time for event-level analysis, supports uniqueness constraints at the race-driver grain, and enables year/round slicing without rejoining raw CSV IDs.

Important Semantics And Sources

- Qualifying status: Derived from presence/absence of Q1–Q3 times (see Status Derivation above). This enables clear counts of DNQ/DNS/DSQ outcomes in analytics.
- Race results status_text: Taken from `status.csv` by joining on `statusid` (not from `results.csv` free-text fields). This standardizes DNF/DNS/DSQ/Finished semantics across seasons.
- Pit stops: Constructor dimension and foreign key are intentionally not present because the source `pit_stops.csv` lacks constructor identifiers; pit stop analytics are by driver, race, circuit, and date.

Observed Statistics During Load

The pipeline logs the distribution of qualifying outcomes during transformation:

2025-10-18 21:03:27 - f1_etl_pipeline.src.transformers.fact_transformer - INFO - Qualifying status distribution:

2025-10-18 21:03:27 - f1_etl_pipeline.src.transformers.fact_transformer - INFO - DNQ: 4,491 records

2025-10-18 21:03:27 - f1_etl_pipeline.src.transformers.fact_transformer - INFO - OK: 3,629 records

2025-10-18 21:03:27 - f1_etl_pipeline.src.transformers.fact_transformer - INFO - DSQ: 2,218 records

2025-10-18 21:03:27 - f1_etl_pipeline.src.transformers.fact_transformer - INFO - DNS: 156 records

ETL Load Results (Per Schema)

Pit stops

2025-10-19 16:07:35 - f1_etl_pipeline - INFO - Pit_stops records processed: 11,371

2025-10-19 16:07:35 - f1_etl_pipeline - INFO - Dimension records loaded: 3,188

2025-10-19 16:07:35 - f1_etl_pipeline - INFO - Fact records loaded: 11,371

2025-10-19 16:07:35 - f1_etl_pipeline - INFO - Total records loaded: 14,559

2025-10-19 16:07:35 - f1_etl_pipeline - INFO - Total records skipped: 0

Qualifying

2025-10-19 16:11:46 - f1_etl_pipeline - INFO - Qualifying records processed:

10,494

2025-10-19 16:11:46 - f1_etl_pipeline - INFO - Dimension records loaded: 2,275

2025-10-19 16:11:46 - f1_etl_pipeline - INFO - Fact records loaded: 10,494

2025-10-19 16:11:46 - f1_etl_pipeline - INFO - Total records loaded: 12,769

2025-10-19 16:11:46 - f1_etl_pipeline - INFO - Total records skipped: 0

Race final positions

2025-10-19 16:12:33 - f1_etl_pipeline - INFO - Results records processed: 26,668

2025-10-19 16:12:33 - f1_etl_pipeline - INFO - Dimension records loaded: 3,400

2025-10-19 16:12:33 - f1_etl_pipeline - INFO - Fact records loaded: 26,668

2025-10-19 16:12:33 - f1_etl_pipeline - INFO - Total records loaded: 30,068

2025-10-19 16:12:33 - f1_etl_pipeline - INFO - Total records skipped: 91

Why some race_result records were skipped

- Duplicate race–driver combinations: The pipeline enforces a unique grain of one record per driver per race. After mapping to surrogate keys, duplicates on (`dim_race_race_key` , `dim_driver_driver_key`) are dropped, keeping the first occurrence. Typical causes include multiple rows for the same pairing due to reclassifications, penalties, or source duplication. These drops contribute to the “Total records skipped”.
- Invalid/missing references: Prior to deduplication, rows missing valid `driverId` , `constructorId` , or `raceId` (i.e., not present in the corresponding dimension lookups) or with nulls in required columns are filtered out. In practice, this is a very small number compared to duplicates.
- Status handling is not a skip reason: `status_text` is joined from `status.csv` ; unmatched status is allowed and does not cause a skip.

How To Run

Use the instructions in `reports/usage_instructions.md` . Summary:

- Before running, set database credentials in `.env` to match your MySQL instance:

- `DATABASE_USER=`
- `DATABASE_PASSWORD=`
- Create all three schemas in MySQL:
 - `mysql -u <user> -p < schemas/qualification_sessions_schema.sql`
 - `mysql -u <user> -p < schemas/pit_stop_schema.sql`
 - `mysql -u <user> -p < schemas/race_final_positions_schema.sql`
- Install dependencies: `pip install -r requirements.txt`
- - Ensure CSVs are in `data/` : `qualifying.csv` , `pit_stops.csv` , `results.csv` , `status.csv` , `circuits.csv` , `constructors.csv` , `drivers.csv` , `racers.csv` .
- Run by schema:
 - Qualifying: `python run_etl.py --schema qualifying`
 - Pit stops: `python run_etl.py --schema pit_stop`
 - Race results: `python run_etl.py --schema race_results`
- Options:
 - Samples: `-sample-size N` (e.g., 1000)
 - Faster: `-skip-validation`

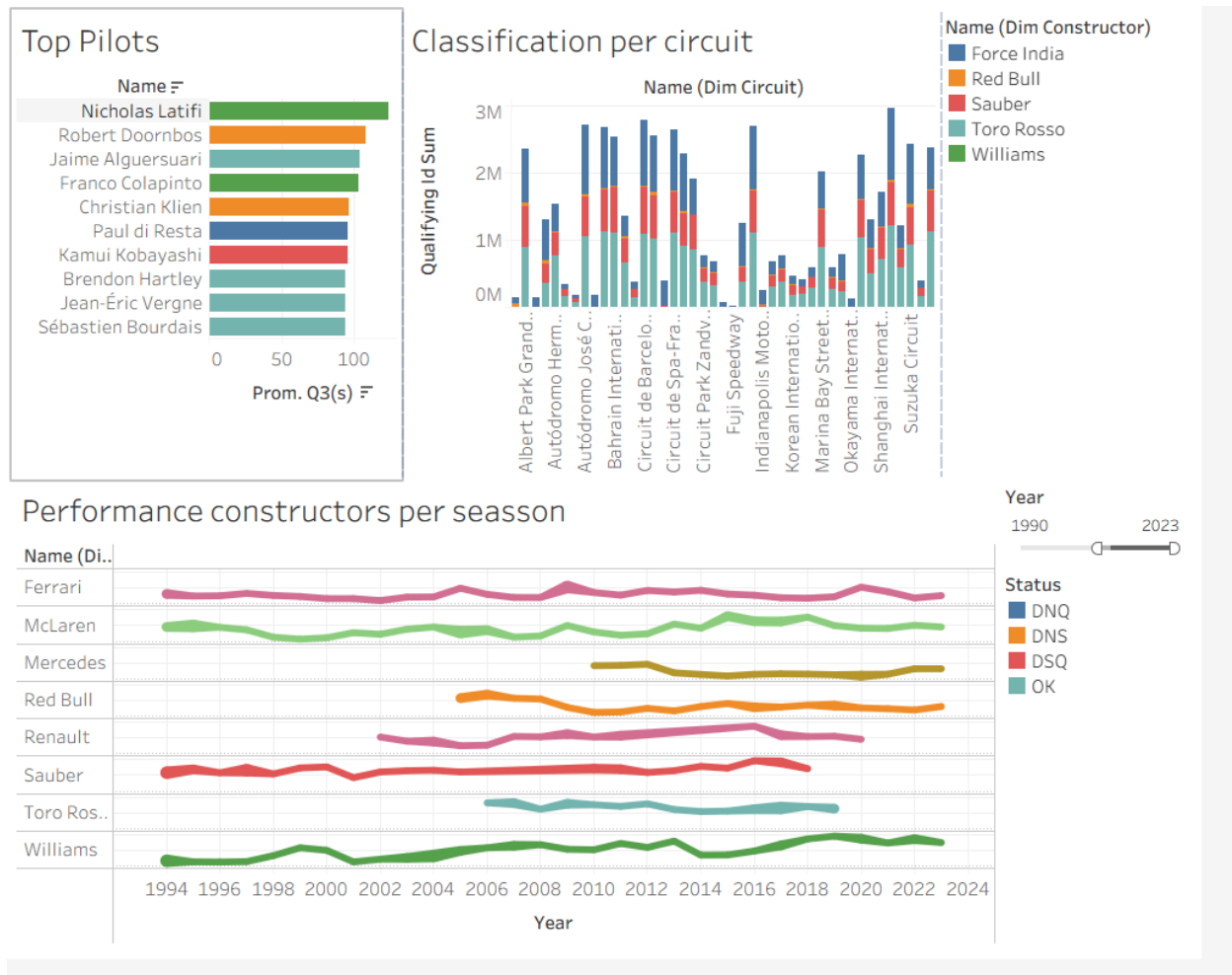
References

- Flow and sequence diagrams: `diagram.md`
- Schema DDLs: `schemas/qualification_sessions_schema.sql` , `schemas/pit_stop_schema.sql` , `schemas/race_final_positions_schema.sql`
- Pipeline entrypoint: `run_etl.py`

Data analysis

In this section we are gonna briefly explain each of the dashboards we made in Tableau (one per squema) with the corresponding screenshots of each. The .twb file will be also submitted.

1.- Qualifying sessions Dashboard:

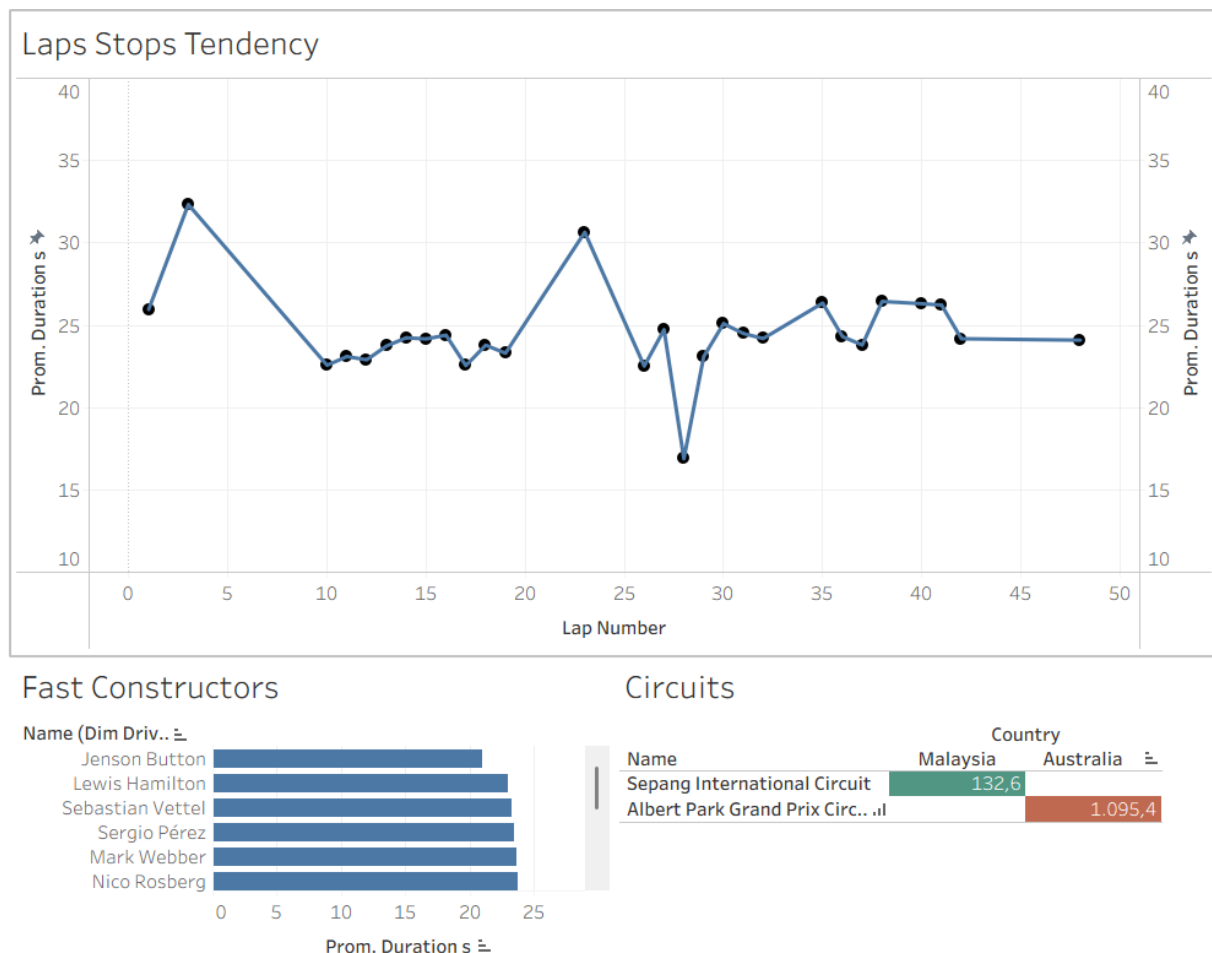


As we can see, we have at the right upper corner, a bar graph that shows the average of the time of q3 (in seconds) for each driver we have data from. We have also included a differentiation per color depending on the constructor name the driver belongs to. This graph allow us to know quickly which drivers get the most competitive times in q3, but also if a specific constructor domains the table.

On the right part of the dashboard, we have another bar graph (this time, a vertical one) at which we can see, for each of the circuits, a sum of qualifying ids, also differentiated in terms of color depending of each of the different status we have. Its function is essentially revealing circuits with frequent incidents (status DNQ, DNS, DSQ) compared to those with greater stability (status OK), helping to detect problematic classification scenarios.

Last, at the bottom of the dashboard, we have a line graph in which we show, for the 8 constructors with the most amount of records, an average of the positions its drivers got at each year. Also, we have added a size classification by which the widest segments of the lines mean the ones with the most amount of drivers at each constructor. With this graph we can see high or low tendencies in terms of the performance (regarding positions) of each F1 team, so we can also compare competitive and non competitive stages for each.

2.- Pit stops Dashboard:

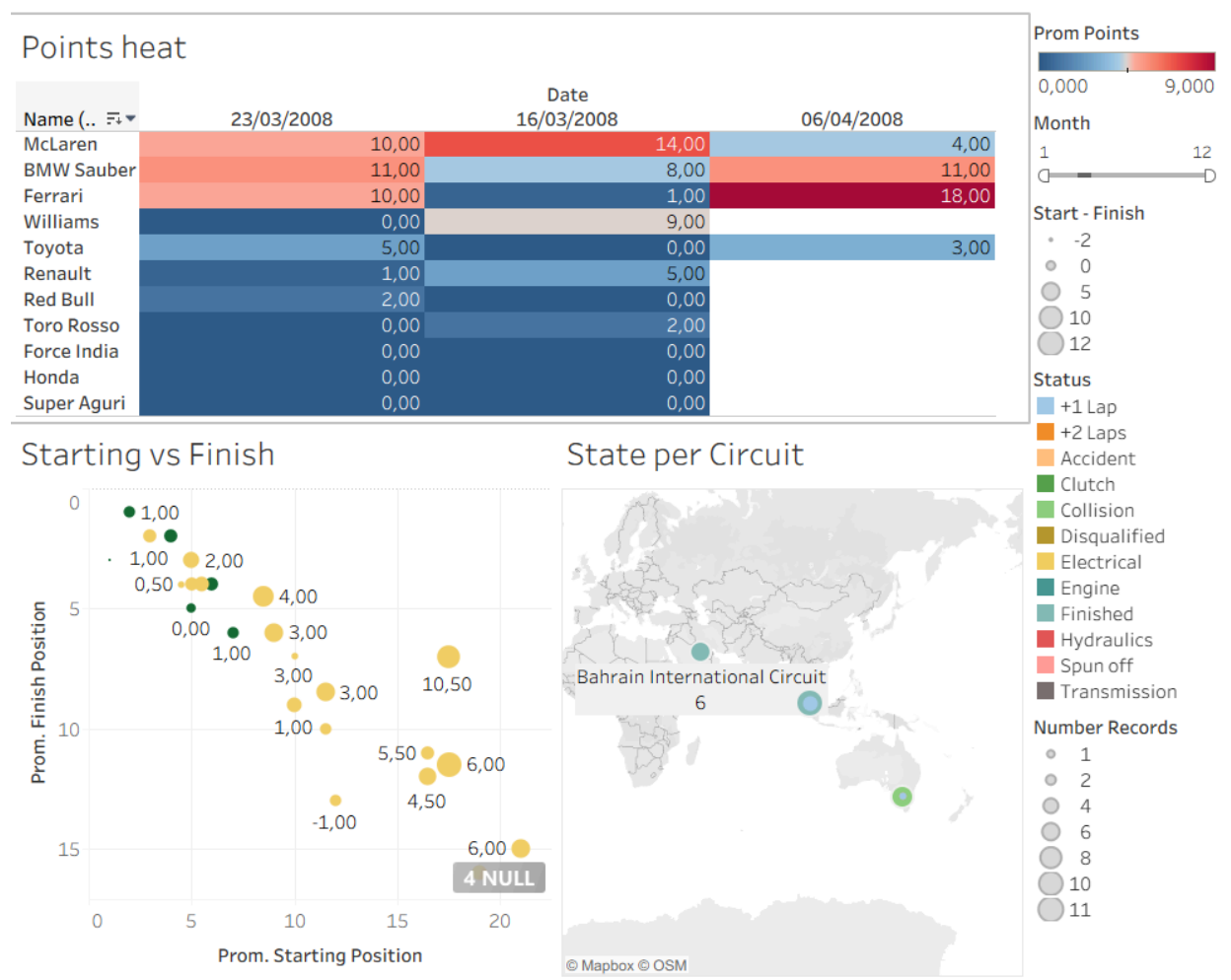


The second dashboard is some way simpler compared to the first one, given that we had less data to play with and less variability regarding some of the fields.

At the upper side we have a line graph which compares the number of the specific lap with the average of the duration in seconds at the pit stop. We can then know the variations between laps and where the average-duration peaks are concentrated.

At the bottom and left side of the dashboard we established a bar graph where the drivers are listed in order from shortest to longest average duration, which allows us to quickly identify those who make the quickest pit stops. We also applied the Routine stop filter, which only keeps stops shorter 100 s, eliminating atypical records.

3.- Race results dashboard:



Regarding the third graph (bounding to the model of race results) we have first a heat table in which we can see (for the only three dates we have data on) the summations of the points gained per each of the teams. We also have a colour classification, where the hottest cells are the ones with the biggest average of points gained on that date, and the coldest cells are the one reflecting the less amount of points gained on average. With a quick look we can then see the evolution of each team during the different dates regarding points gained.

We also have a disperssion graph in which we can see (with the cursor on each dot) which driver has a specific starting and finishing position on average. The graph is ordered regarding the difference in between starting and finishing positions, so that the points above the diagonal represents drivers which made a comeback (starting positions > finishing position). We also made a colour classification, but just on the months on the race (we didn't have so much variability regarding dates of the races), so yellow points represent results on March, and green points represent results on April.

At last, we have a map for the circuits we have data on our race_results squema, so we just have marked these three circuits (Bahrain International Circuit at the Left, Sepang International Circuit at the middle, and Albert Park Grand Prix Circuit at the right side). The size of the dots also reflects the amount of records per circuit, and the color also allow us to differentiate in between different status (considering also the use of the filter at the right side).